Assignment 4: Build Non-Linear Models Part 2

Juan Maldonado Franco
DDS-8555 Predictive Analysis
Mohamed Nabeel

In [1]:
```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

ROOT = Path.cwd()
for parent in [ROOT, *ROOT.parents]:
    if (parent / "DDS-8555 - Predictive Analysis").exists():
        COURSE = parent / "DDS-8555 - Predictive Analysis"
        break
else:
    COURSE = ROOT.parents[1]
DATA = COURSE / "data"
KAGGLE = DATA / "kaggle"
SUBMISSIONS = DATA / "submissions"
RANDOM_STATE = 42
pd.set_option("display.max_columns", 80)
```

Conceptual Question 3

The fitted curve is f(X) = 1 + X - 2(X - 1)^2I(X >= 1). For X < 1, the indicator is zero, so the curve is the line f(X) = 1 + X. For X >= 1, the curve becomes 1 + X - 2(X - 1)^2, which is continuous at X = 1 but bends downward after the knot. The slope is 1 before the knot and 1 - 4(X - 1) after the knot. The curve therefore rises linearly until X = 1, then becomes concave downward and eventually declines.

Applied Question 8: Nonlinear Patterns in Auto

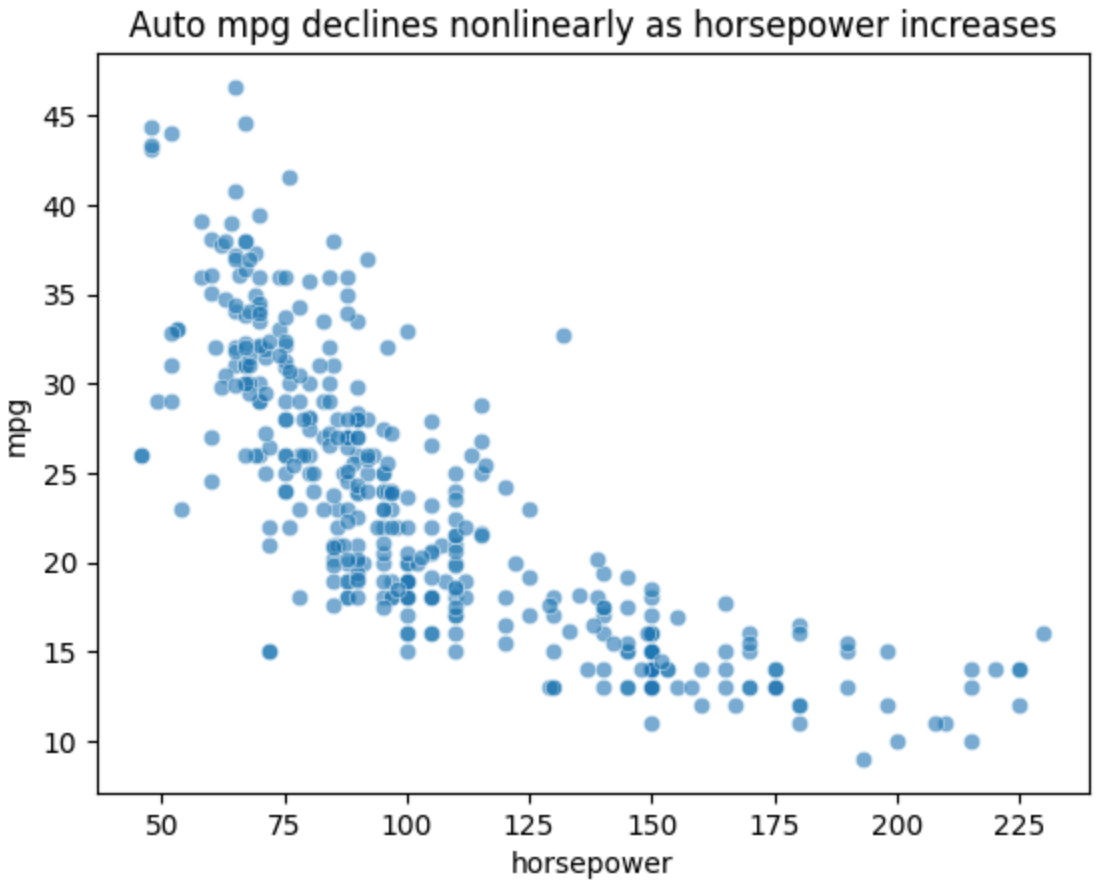
The Auto data set is used to test whether nonlinear transformations improve prediction of miles per gallon. This is a practical question because vehicle efficiency often changes nonlinearly with weight, displacement, horsepower, and model year.

In [2]:
```
from ISLP import load_data
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

auto = load_data("Auto").dropna()
features = ["horsepower", "weight", "displacement", "acceleration"]
X = auto[features]
y = auto["mpg"]
kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
rows = []
for degree in range(1, 5):
    model = Pipeline([("poly", PolynomialFeatures(degree=degree, include_bias=False
    rmse = (-cross_val_score(model, X, y, cv=kf, scoring="neg_root_mean_squared_err
    rows.append({"degree": degree, "cv_rmse": rmse})
display(pd.DataFrame(rows))
sns.scatterplot(data=auto, x="horsepower", y="mpg", alpha=.6)
plt.title("Auto mpg declines nonlinearly as horsepower increases")
plt.show()
```

	degree	cv_rmse
0	1	4.290986
1	2	4.108434
2	3	4.337040
3	4	17.311932



Kaggle Nonlinear Models

The competition portion uses gradient boosting and a random forest. Both models introduce nonlinearity through tree splits rather than through manually specified polynomial terms or splines.

In [3]:
```
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.metrics import mean_squared_log_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

abalone = pd.read_csv(KAGGLE / "playground-series-s4e4" / "train.csv")
X = abalone.drop(columns=["Rings"])
y = abalone["Rings"]
pre = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), ["Sex"])],
X_train, X_valid, y_train, y_valid = train_test_split(X.drop(columns=["id"]), y, te
models = {
    "Gradient boosting": Pipeline([("pre", pre), ("model", GradientBoostingRegresso
    "Random forest": Pipeline([("pre", pre), ("model", RandomForestRegressor(n_esti
}
rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = np.maximum(model.predict(X_valid), 1)
    rows.append({"model": name, "validation_rmsle": np.sqrt(mean_squared_log_error(
pd.DataFrame(rows)
```

Out[3]:

	model	validation_rmsle
0	Gradient boosting	0.155624
1	Random forest	0.154466

In [4]:
```
from pathlib import Path
status_path = SUBMISSIONS / "kaggle_submission_status_playground-series-s4e4.txt"
text = status_path.read_text(encoding="utf-8", errors="ignore")
print("\n".join([line for line in text.splitlines() if "A4_" in line or "fileName"
```

Interpretation

The Auto exercise supports nonlinear modeling because polynomial terms reduce cross-validated error relative to a strictly linear form. For Abalone, gradient boosting produced the strongest Kaggle score among the two submitted nonlinear models. The assumption investigation shifts from normal residual theory to validation behavior, residual structure, and overfitting control because tree ensembles do not require the same parametric residual assumptions as ordinary least squares.

References

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189-1232. https://doi.org/10.1214/aos/1013203451

Hastie, T., & Tibshirani, R. (1990). *Generalized additive models*. Chapman and Hall.

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. https://doi.org/10.1007/978-3-031-38747-0